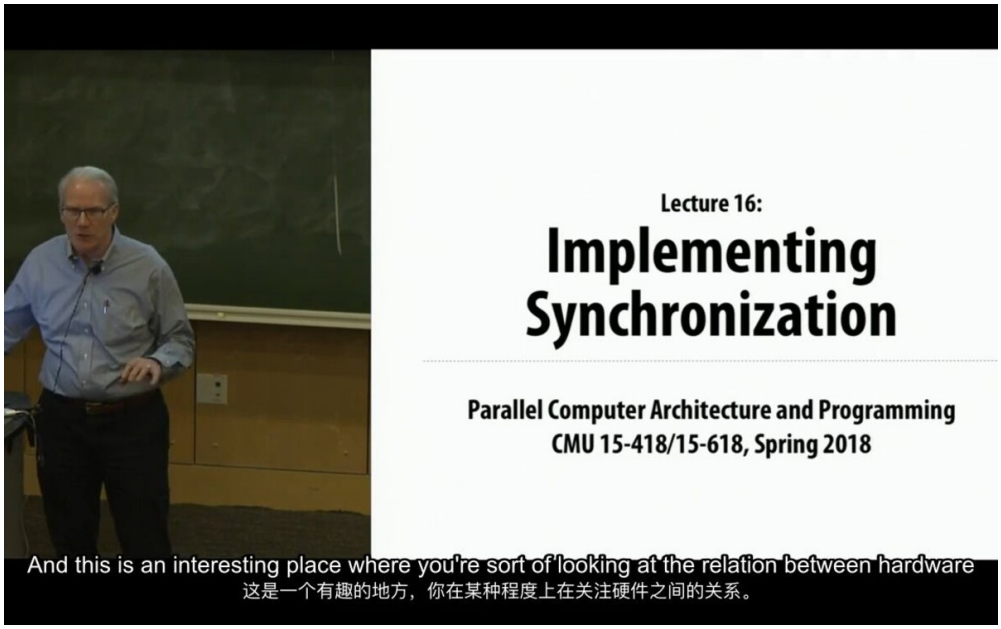
课程笔记

同步的实现：自旋锁、原子原语与屏障

CMU 15-418/618 并行计算机架构与编程 · Lecture 20（Spring 2018）

lecture-to-notes

2026 年 10 月 28 日



视频作者/频道：CMU 15-418/618（B 站搬运：alexphil）
发布日期：2018 年春季学期
视频时长：1 小时 5 分 45 秒
视频链接：`https://www.bilibili.com/video/BV1au4y1i7DD/`

目录

1 同步原语的实现：从线程映射说起 2
1.1 硬件线程与活跃线程 . . . 2
1.2 操作系统做了什么 . . . 3
1.3 本章小结 . . . 4

2 同步事件的三个阶段 5
2.1 获取、等待、释放 . . . 5
2.2 忙等待 . . . 5
2.3 但在并行计算里它未必坏 . . . 6
2.4 阻塞式等待 . . . 7
2.5 本章小结 . . . 8

3 一个看似正确却有缺陷的自旋锁 8
3.1 朴素实现 . . . 8
3.2 缺陷在哪 . . . 9
3.3 但流量问题很严重 . . . 9
3.4 本章小结 . . . 10

4 硬件原语：test-and-set 与 cmpxchg 10
4.1 test-and-set . . . 10
4.2 锁应独占一个缓存行 . . . 10
4.3 一致性流量分析 . . . 11
4.4 x86 的 cmpxchg . . . 12
4.5 本章小结 . . . 12

5 理想锁的四个特征 13
5.1 本章小结 . . . 13

6 三个改进方案 14
6.1 test-and-test-and-set . . . 14
6.2 退避 . . . 15
6.3 票号锁 . . . 15
6.4 本章小结 . . . 16

7 屏障与后续 16
7.1 集中式屏障 . . . 16
7.2 预览：事务内存 . . . 17
7.3 本章小结 . . . 17

8 总结与延伸 18

1 同步原语的实现：从线程映射说起

讲者开场先回顾线程如何映射到硬件，因为同步的开销与这个映射直接相关。

1.1 硬件线程与活跃线程

一个处理器有若干核，每个核支持若干个执行上下文。Intel 把这个叫超线程，正式名称是同时多线程。典型情况下每个核两个。

因此在任一时刻，硬件能同时处理的线程数上限就是执行上下文的数量。

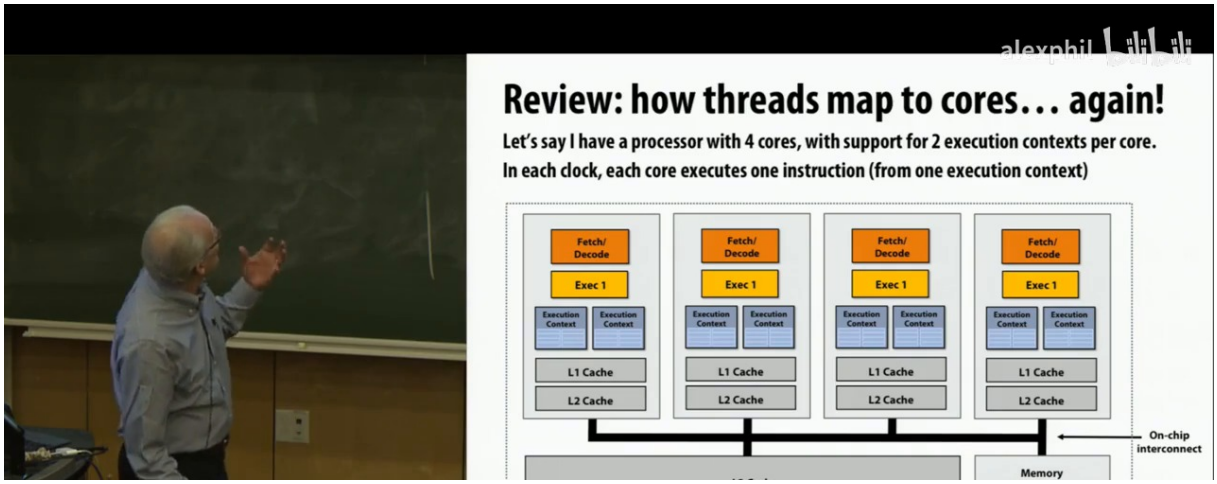


图 1: 回顾：线程如何映射到核心与执行上下文[1]

但若在机器上跑 top 或列出活跃进程，通常会发现多得多的线程。讲者用一台笔记本举例：浏览器、各种主动使用的程序，加上 Dropbox 之类被动运行的服务——它们都在后台保持进程，以便有事发生时能响应。

于是可能有 200 个以上的潜在活跃线程，远多于可执行的硬件线程数。

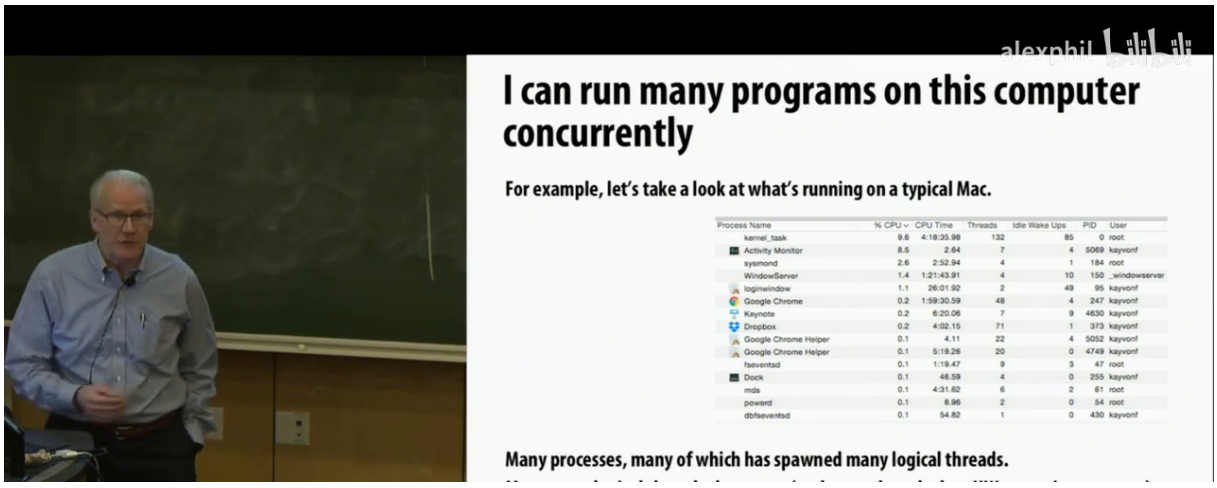


图 2: 一台机器上可以并发运行许多程序[2]

[1] 视频画面时间区间：00:12:00–00:12:15。若为裁剪图，时间区间仍对应原视频画面。

[2] 视频画面时间区间：00:13:00–00:13:15。若为裁剪图，时间区间仍对应原视频画面。

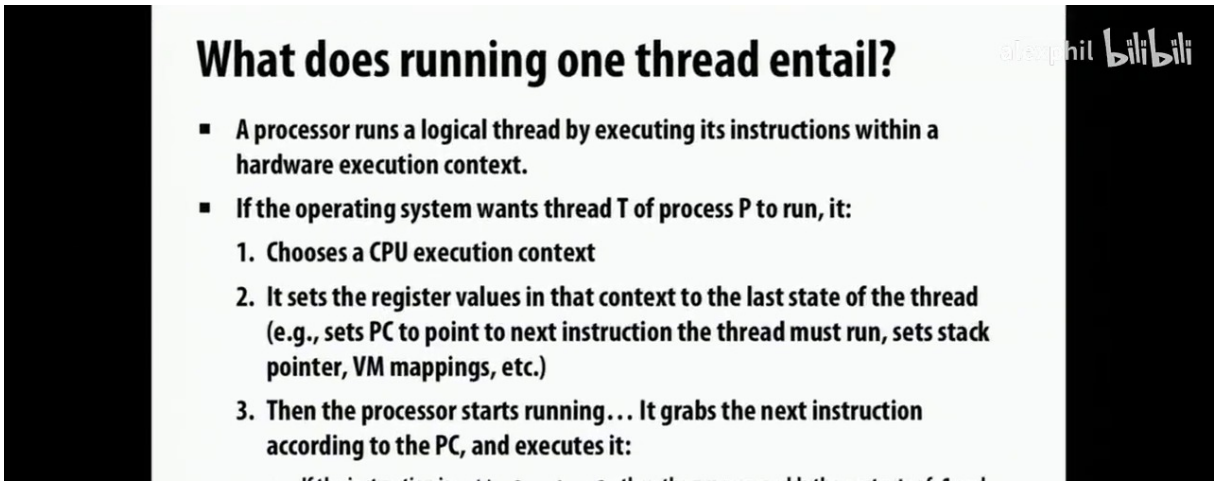


图 3: 运行一个线程意味着什么：处理器执行其指令流[3]

1.2 操作系统做了什么

操作系统必须决定哪些线程应该运行。它会挑一个核、挑该核上的一个执行上下文，把线程分配过去。这涉及从存储的状态中取出寄存器值。

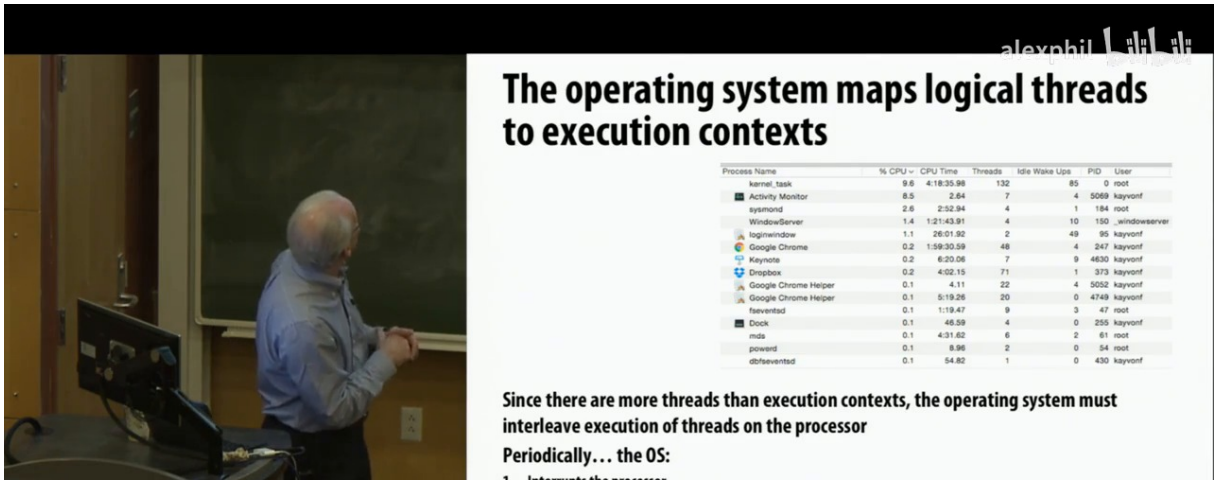


图 4: 操作系统把逻辑线程映射到执行上下文[4]

[3]视频画面时间区间：00:14:00–00:14:15。若为裁剪图，时间区间仍对应原视频画面。

[4]视频画面时间区间：00:15:30–00:15:45。若为裁剪图，时间区间仍对应原视频画面。

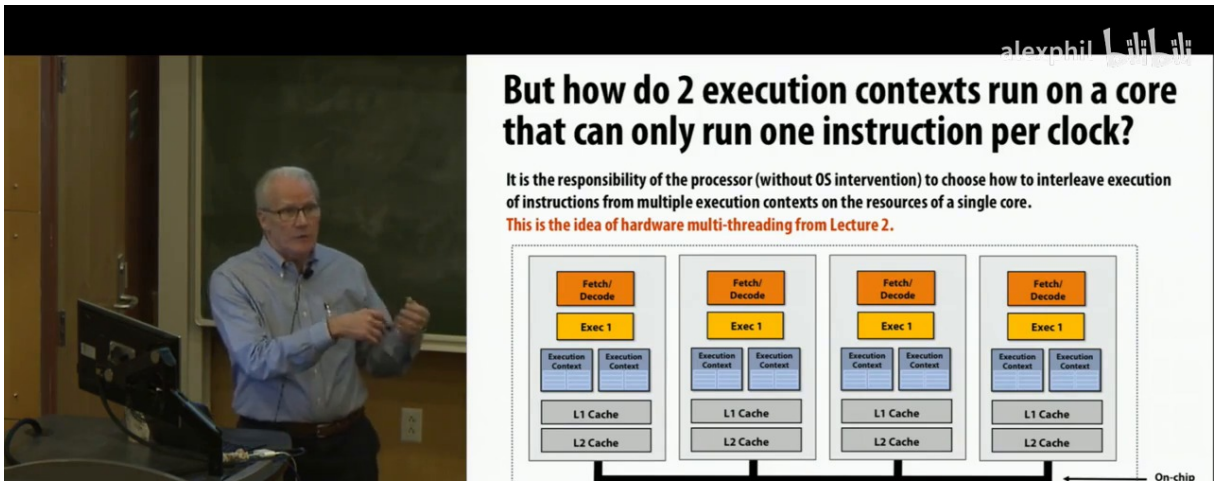


图 5: 两个执行上下文如何在一个每周期只能发一条指令的核上运行[5]

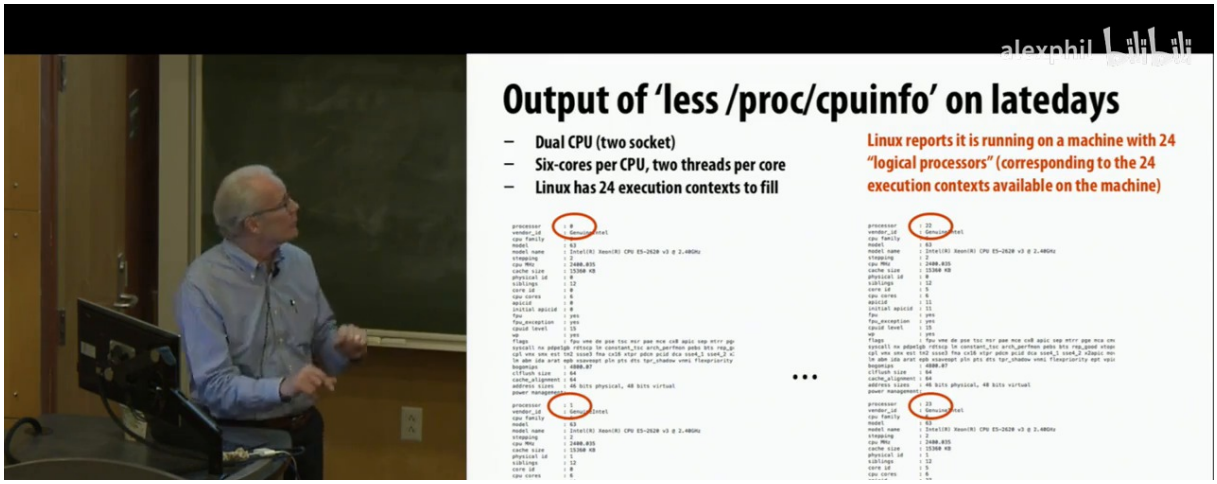


图 6: late days 机器上 /proc/cpuinfo 的输出：双插槽[6]

同步开销的量级由"是否牵涉操作系统"决定

纯硬件的原子操作在几十到几百个周期量级，而任何涉及操作系统调度的操作（把线程放入等待队列、切换上下文）在数千周期量级。两者相差一到两个数量级，因此选择忙等待还是阻塞，本质上是在选择付出哪一档开销。

1.3 本章小结

硬件线程数远少于活跃线程数，操作系统负责把线程映射到执行上下文。同步的开销量级取决于是否牵涉操作系统：纯硬件是几十到几百周期，阻塞式是数千周期。这个差距决定了后面所有设计取舍。

下一章看同步事件本身包含哪几个阶段。

[5] 视频画面时间区间：00:16:00–00:16:15。若为裁剪图，时间区间仍对应原视频画面。

[6] 视频画面时间区间：00:17:00–00:17:15。若为裁剪图，时间区间仍对应原视频画面。

2 同步事件的三个阶段

2.1 获取、等待、释放

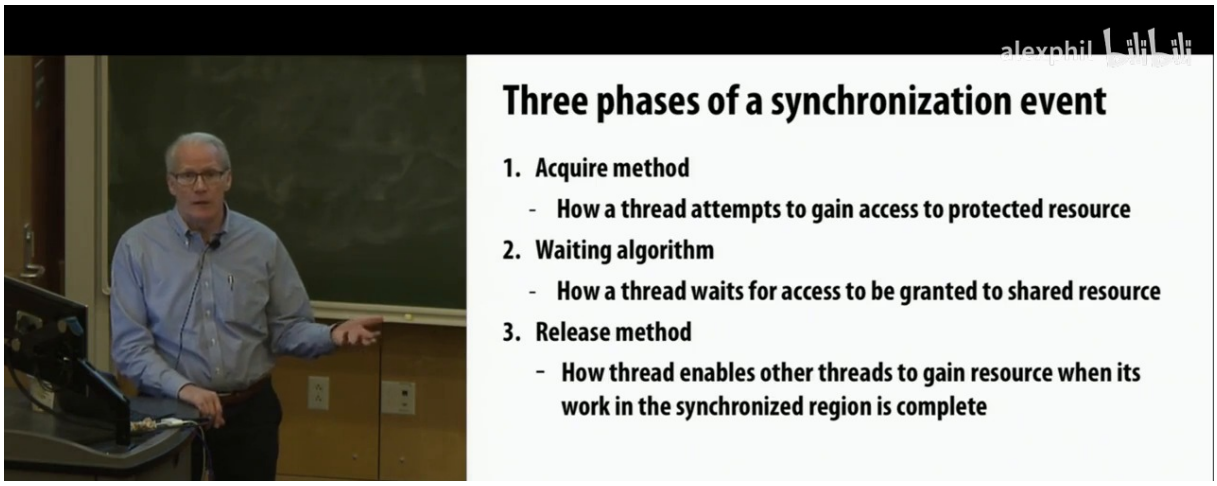


图 7: 同步事件的三个阶段：获取方式、等待期间做什么、释放方式[7]

讲者把一次同步拆成三段。第一是获取：一个线程如何尝试取得锁或某种访问受保护资源的许可。第二是等待：如果没能取得，在等待期间做什么。第三是释放：用完这段临界区之后，如何把锁交出去。

三阶段框架让比较变得可比

不同锁实现在获取方式上可能差别很大，但等待与释放的方式往往决定了它在竞争下的表现。把实现拆成三段后，就可以分别问：获取在无争用时是否够快、等待是否产生流量、交接是否公平。这比笼统比较”哪个锁更快”更有信息量。

这个三段划分是后面所有比较的框架。

2.2 忙等待

最朴素的做法是忙等待，也叫自旋：在循环里检测条件，不成立就再测一次。释放则是把条件置为真，简单得几乎不需要解释。

[7] 视频画面时间区间：00:19:00–00:19:15。若为裁剪图，时间区间仍对应原视频画面。

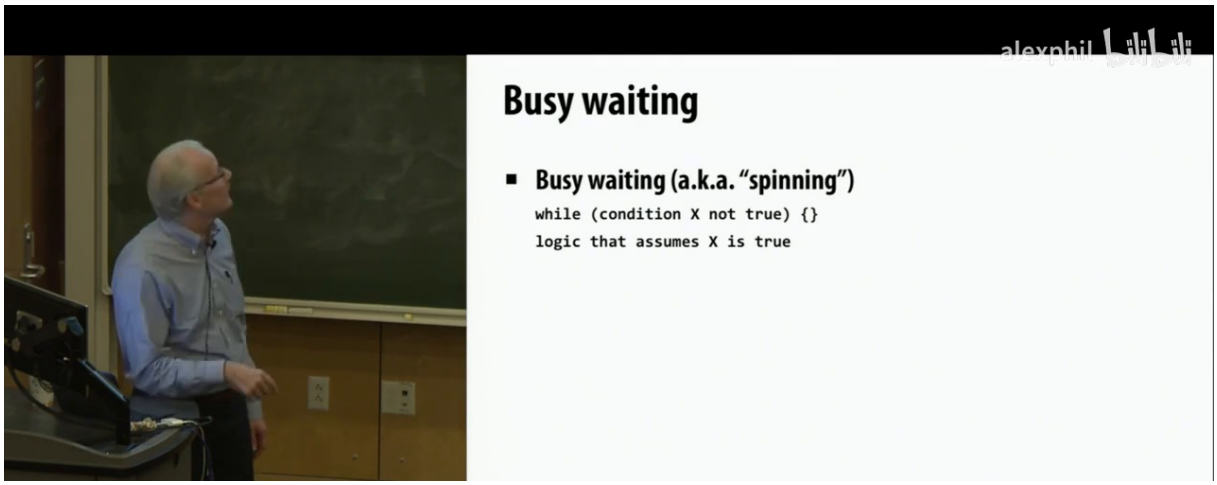


图 8: 忙等待：循环检测条件直至成立[8]

你可能听说过忙等待是坏事——这大体上正确，因为处理器在那个循环里没做任何有用的事，只是绕回自己。

2.3 但在并行计算里它未必坏

讲者给出一个重要的限定：在高并行、以性能为导向的系统中，忙等待没那么糟。因为如果你没在等待，本来也没有别的事可做。

他举例说，我们试图用满全部十二个处理器；如果不执行应用代码，那确实没什么有用的活可干。

因此自旋的机会成本可以写成：

$$\mathrm{cost}_{\mathrm{spin}} = \begin{cases} 0, \& \text{若没有其他可运行线程} \\ T_{\mathrm{wait}} \cdot \text{可复用算力}, \& \text{否则} \end{cases}$$

- 在并行计算程序里通常落在第一种情形，因为所有核都已被占满；
- 在多用户系统里落在第二种情形，自旋挤占了本可给别人用的时间片。

而且忙等待有个明确的优势：等待与释放的开销都很低。

[8] 视频画面时间区间：00:19:30–00:19:45。若为裁剪图，时间区间仍对应原视频画面。

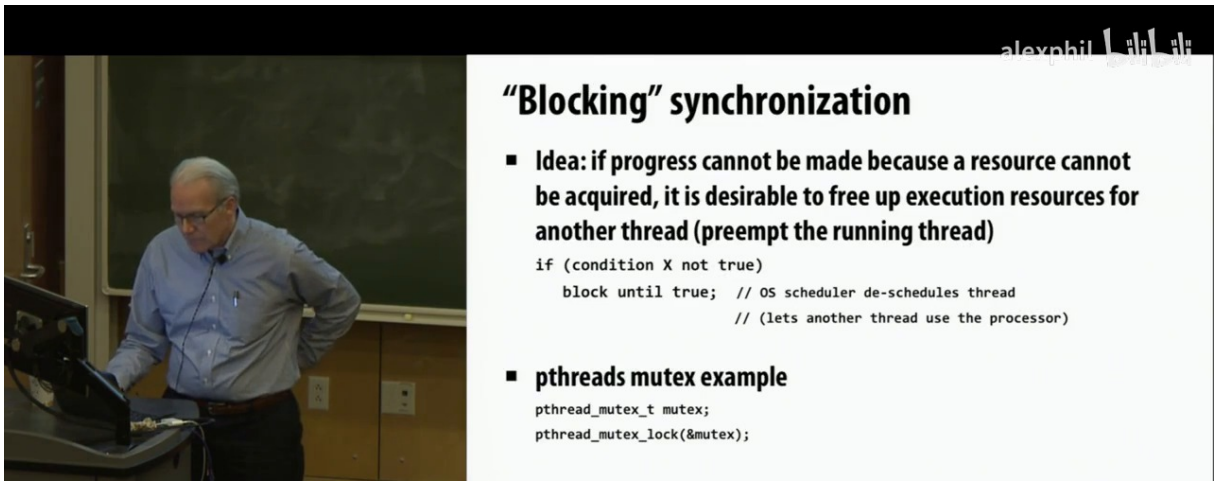


图 9: 阻塞式同步：若资源无法获取则让出控制权[9]

2.4 阻塞式等待

另一种做法是：如果等待的条件不成立，就交出对这些硬件资源的控制权，把当前线程放到一边，换一个已就绪的线程进来执行。

讲者说在使用 pthreads 互斥量时就是这个行为——线程要么自旋，要么阻塞。

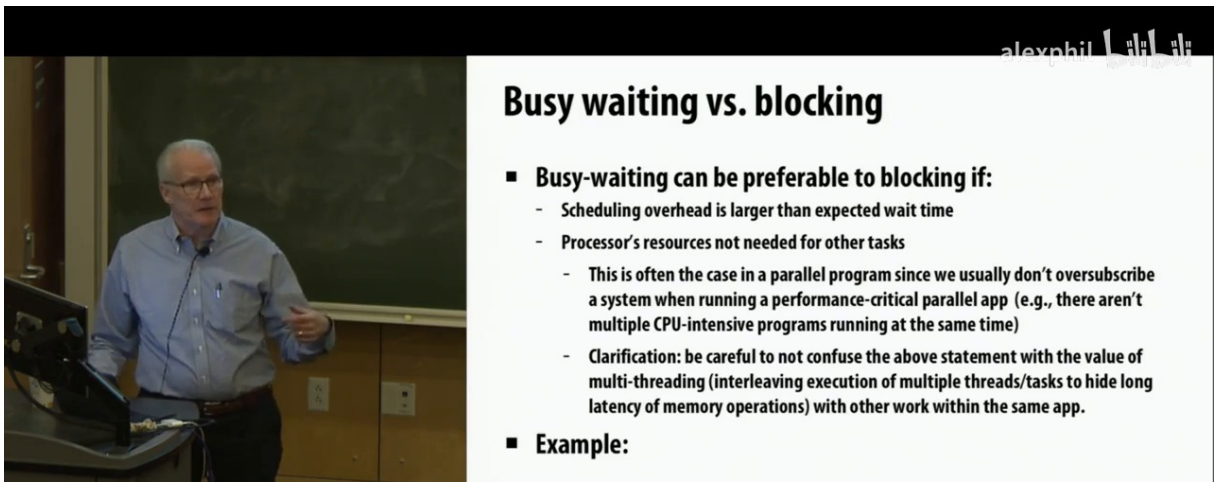


图 10: 忙等待与阻塞的对比：开销量级差异明显[10]

在被许多用户或许多类型进程共享的系统里，阻塞显然更可取，但它有不小的开销。忙等待完全不需要操作系统介入，是纯硬件的事；阻塞则需要被操作系统放入某个队列。

于是量级差别出来了：任何涉及操作系统的操作都在 2000 个时钟周期以上；纯硬件的操作只需 50 到 200 个周期。

[9] 视频画面时间区间：00:21:00–00:21:15。若为裁剪图，时间区间仍对应原视频画面。

[10] 视频画面时间区间：00:22:00–00:22:15。若为裁剪图，时间区间仍对应原视频画面。

忙等待的适用条件

讲者给出的判据其实很明确：若处理器除了等待没有别的活可干，自旋的代价只是浪费算力而不浪费别人的时间；若系统里还有别的线程可以推进，自旋就浪费了本可复用的执行资源。因此并行计算程序倾向自旋，而通用操作系统倾向阻塞。

2.5 本章小结

同步分获取、等待、释放三阶段。忙等待是循环检测，开销极低，在高并行计算中并不坏，因为等待时本无别事可做。阻塞式等待让出控制权，适合多用户共享系统，但需操作系统介入，开销高数千倍。

这个框架可以用来分析任何锁的实现。

3 一个看似正确却有缺陷的自旋锁

3.1 朴素实现



图 11: 热身：一个简单但有缺陷的自旋锁[11]

讲者用一段伪汇编说明。加锁部分：从内存把共享变量读入寄存器，与零比较；若非零则跳回开头继续自旋；若为零则把该内存位置置为一。解锁就是把内存置零，好让其他在自旋的程序看到变化、抢到它并再次置一。

讲者说大多数人已经知道这段代码有 bug。他的提问得到的回答是：它不是原子的。

[11] 视频画面时间区间：00:23:00–00:23:15。若为裁剪图，时间区间仍对应原视频画面。

3.2 缺陷在哪

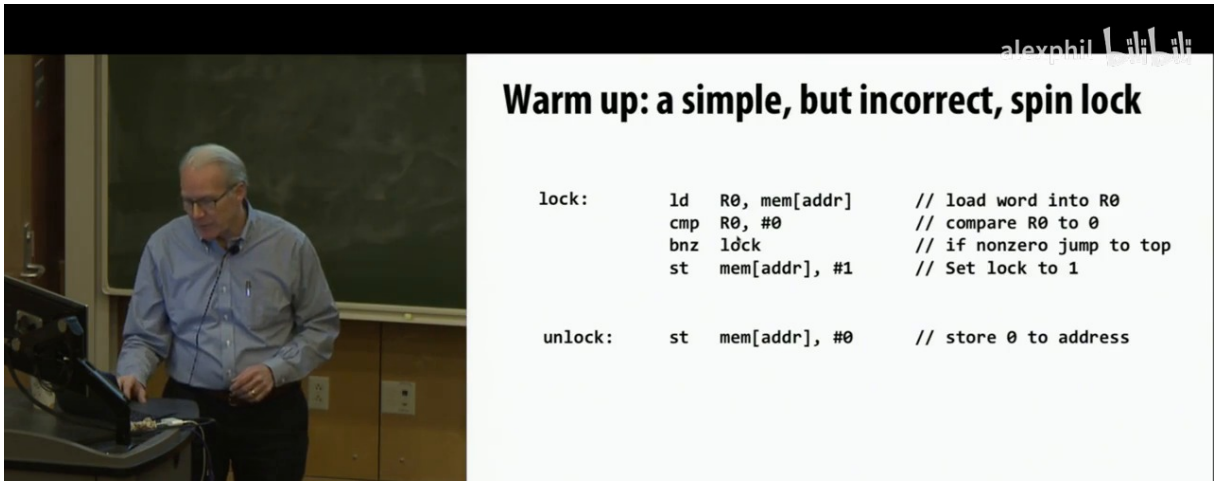


图 12：缺陷所在：测试与置位之间存在可被插入的窗口[12]

问题正是这个：别的进程可能恰好插进中间。你做完测试、发现是零，正准备写一，而另一个进程同时也这么想。于是可能有多个线程一起通过这道锁。

讲者补充说现实中不太容易发生，因为——现在你们已经看过缓存协议了——尽管内存操作相当快，但要作废共享副本、把它们推出去、再取回一份，是件不小的工程。所以实际出错概率不高。

这个窗口的大小可以粗略表示为：

$$T_{\text{window}} \approx T_{\text{load}} + T_{\text{invalidate}} + T_{\text{transfer}}$$

- T_{load}：读取锁值的时间；
- $T_{\text{invalidate}}$：作废其他持有者副本的时间；
- T_{transfer}：把缓存行取回本地的时间；
- 窗口越长，其他线程插入的机会越大。

因此朴素自旋锁的安全性依赖一个未声明的假设：窗口足够短。这个假设在核数增加时会失效。

3.3 但流量问题很严重

讲者的分析很关键：若一堆进程在跑同一段代码、都在等同一把锁，它们会把这份锁的只读副本拉进各自缓存。于是自旋会跑得飞快，几乎不产生总线流量。

但一旦解锁发生，就会有一场巨大的争夺，所有等待者都要去更新它。因此确实很可能出现同步错误。

低出错概率不等于正确

”实际不太容易发生”是缓存协议延迟带来的副作用，而不是任何保证。处理器数量增加、缓存一致性实现变化、或编译器重排都可能让这个窗口变得更容易命中。依赖概率来保证正确性在并发代码中是不可接受的——必须用原子原语。

[12] 视频画面时间区间：00:25:00–00:25:15。若为裁剪图，时间区间仍对应原视频画面。

3.4 本章小结

朴素自旋锁用普通读写实现，测试与置位之间存在窗口，多个线程可能同时通过。实际出错概率因缓存协议的开销而降低，但这不构成保证。更严重的是解锁瞬间所有等待者一起争抢，产生大量一致性流量。

这引出对硬件原子原语的需求。

4 硬件原语：test-and-set 与 cmpxchg

4.1 test-and-set

多数处理器提供了专门为此设计的指令，最经典的是 test-and-set。

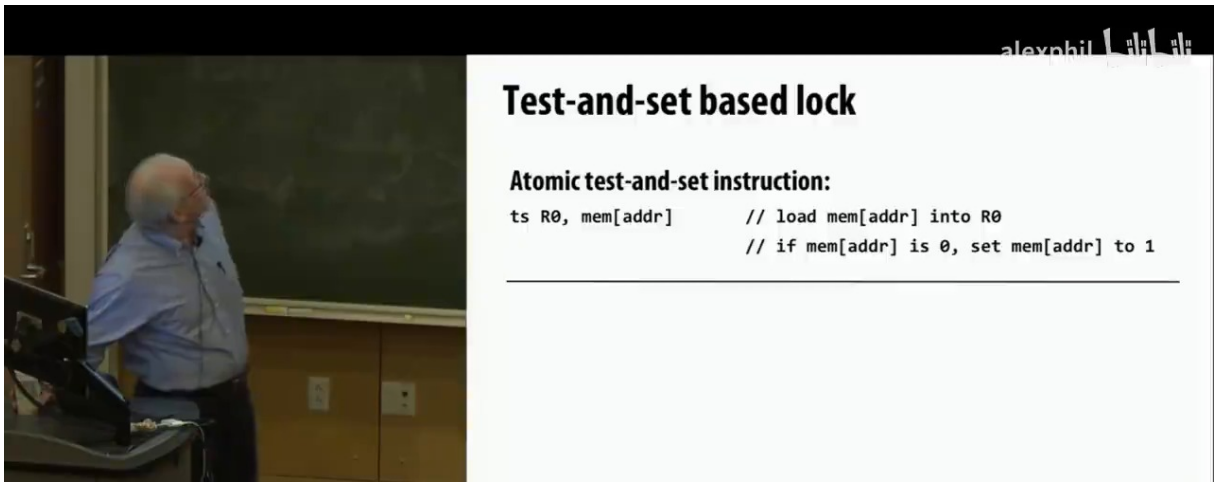


图 13: 基于 test-and-set 的锁：原子地读取并把零置一[13]

它的语义是：原子地把一个内存位置读入寄存器，若该值为零则置一。还有各种变体，但经典形式就是这样。

讲者强调这些语义非常专用，显然只对同步有用。关键点是它原子——读、测试、写永远是一个不可中断的动作，读写被保证锁在一起。事后可以通过查看寄存器零判断是否成功取得。

原子性保证的是”不可分割”，不是”不会失败”

test-and-set 的原子性确保读与写在硬件层面不被插入，但并发线程仍可能在你之前成功获取锁，此时你读到的是旧值一、获取失败。因此原子指令解决的是正确性，循环重试解决的是进展性——两者缺一不可。

4.2 锁应独占一个缓存行

讲者顺带提到一个实现细节：锁虽然逻辑上只需要一位，但通常实现为一个数据字，而且往往要放在单独的缓存行里。

[13] 视频画面时间区间：00:26:00–00:26:15。若为裁剪图，时间区间仍对应原视频画面。

把锁放在单独缓存行是伪共享的预防措施

若锁与其他频繁访问的变量共享一行，那么每次加解锁都会作废那一行，连带影响其他变量的缓存。把锁独占一行，使加解锁的流量只影响锁本身。这与第 13 讲的伪共享是同一问题的两种表现。

4.3 一致性流量分析

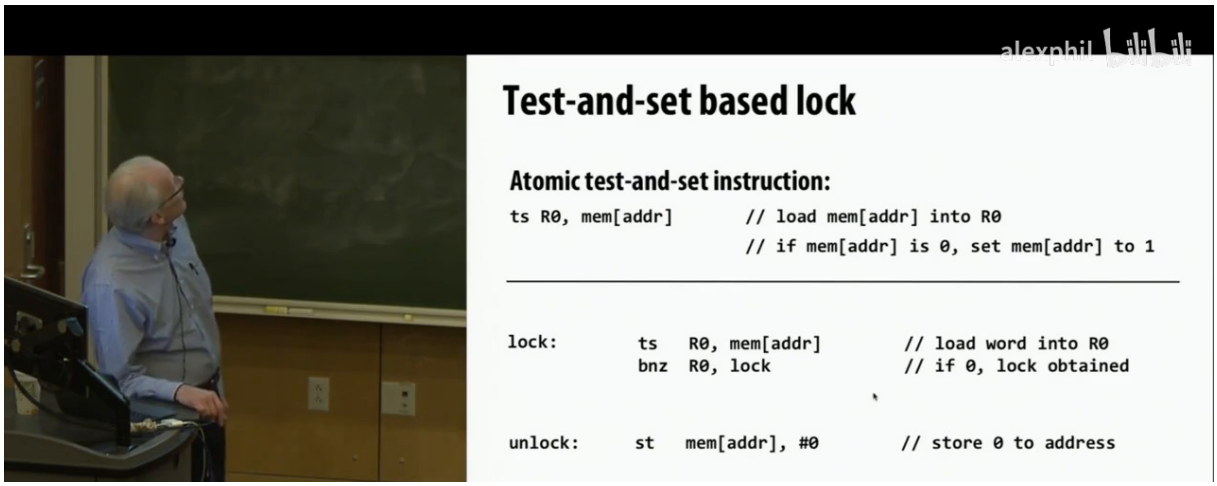


图 14: test-and-set 锁：分析其一致性流量[14]

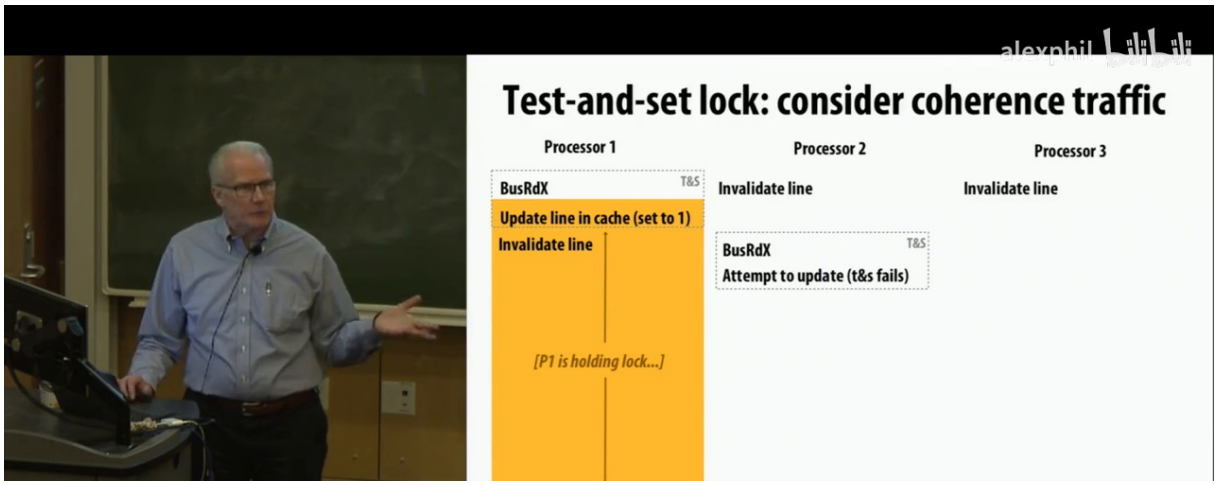


图 15: 流量分析续：解锁时所有处理器都要作废自己的副本[15]

讲者用 3 个处理器的场景分析：每次 test-and-set 都要向总线发出读并申请独占，使其他 2 个持有者的副本作废。

[14] 视频画面时间区间：00:27:30–00:27:45。若为裁剪图，时间区间仍对应原视频画面。
[15] 视频画面时间区间：00:29:00–00:29:15。若为裁剪图，时间区间仍对应原视频画面。

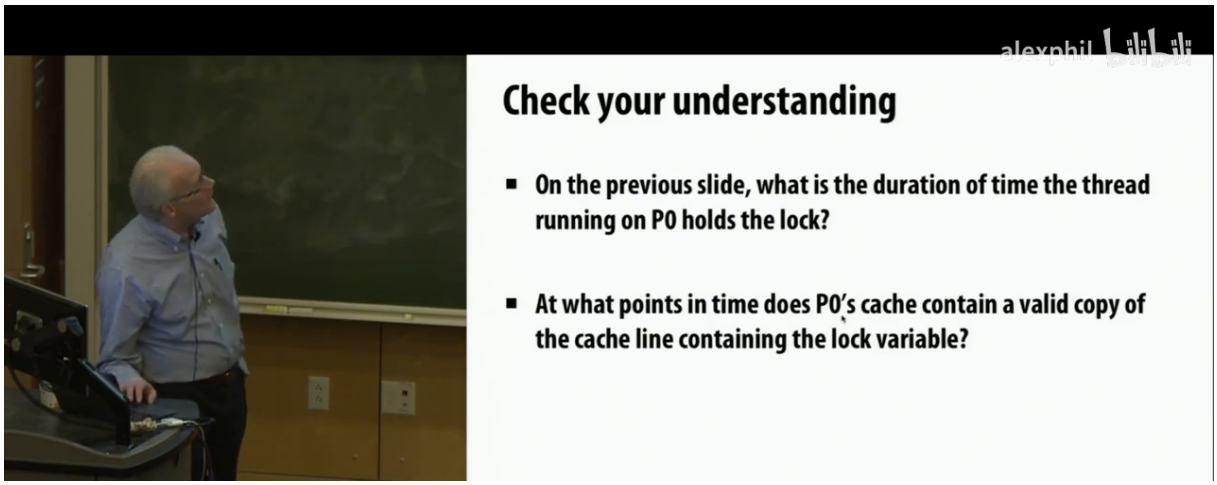


图 16: test-and-set 锁的性能基准：执行 N 次加解锁[16]

4.4 x86 的 cmpxchg

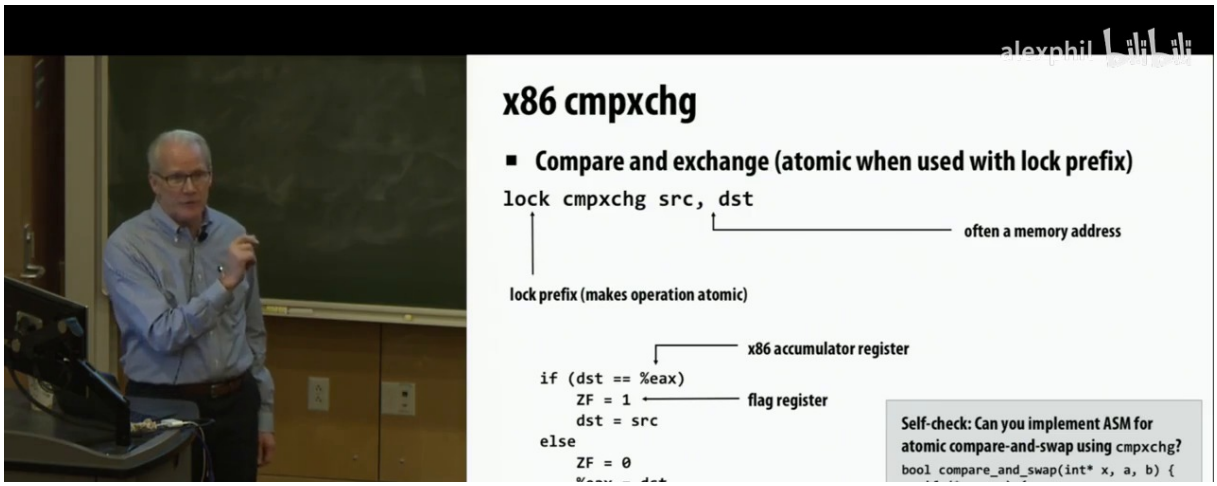


图 17: x86 的 cmpxchg：配合 lock 前缀即为原子比较交换[17]

讲者给出真实指令的例子：x86 的比较并交换。它本身是一个比较交换操作，与 lock 前缀配合使用时才是原子的。

4.5 本章小结

test-and-set 原子地完成读、测试、写三步，锁应独占一个缓存行以避免连带失效。但每次 test-and-set 都要申请独占，解锁时所有等待者一起争抢，产生大量流量。x86 用 cmpxchg 加 lock 前缀提供同类能力。

那么一把” 好锁” 应该满足什么?

[16] 视频画面时间区间：00:31:00–00:31:15。若为裁剪图，时间区间仍对应原视频画面。

[17] 视频画面时间区间：00:33:00–00:33:15。若为裁剪图，时间区间仍对应原视频画面。

5　理想锁的四个特征

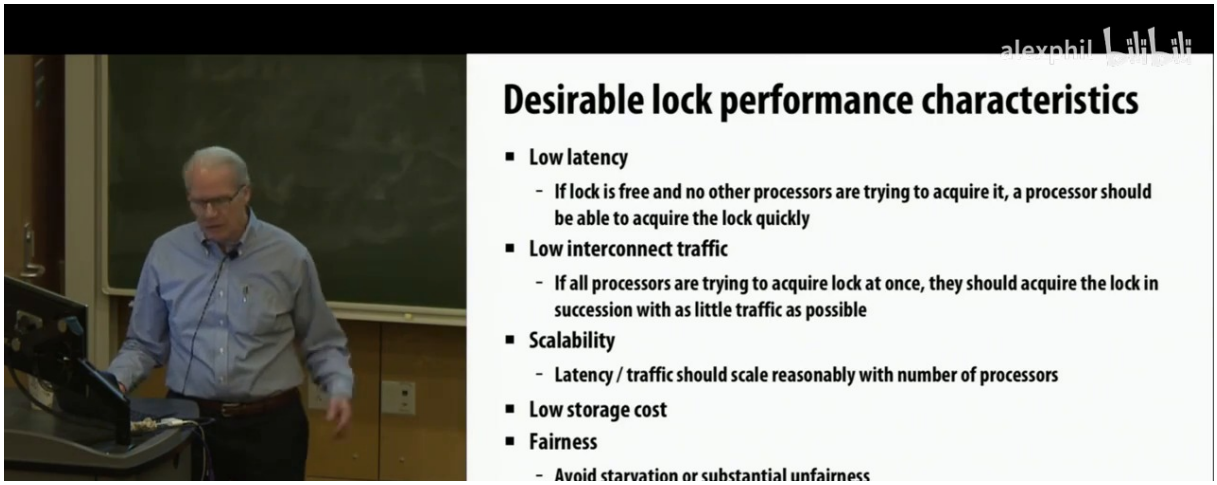


图 18: 理想的锁性能特征：低延迟、低流量、低存储、公平[18]

讲者给出评价锁的四个维度，并逐项对照 test-and-set。

维度	含义	test-and-set 的表现
低延迟	锁空闲且无争用时，加锁应几乎无额外代价	好（等价于一条存储）
低流量	等待与交接不应产生大量互连流量	差（交接时全部争抢）
低存储	每把锁占用的空间应很小	好（一个数据字）
公平	不应让某个线程长期饥饿	差（无顺序保证）

表 1: 理想锁的四个特征与 test-and-set 的对照

讲者的总结是：test-and-set 赢了一些，但不多。

四个维度互相冲突，没有全面最优的锁

低延迟要求简单，公平要求排队，低流量要求错开，低存储要求紧凑。任何改进都是在这些维度间重新分配代价。因此评价一个锁实现必须指明它针对哪种竞争程度与哪种负载模式，而不是笼统地说”这个锁更快”。

5.1　本章小结

理想锁需低延迟、低流量、低存储、公平。test-and-set 在延迟与存储上表现好，在流量与公平上差。这四项互相冲突，因此改进方案都是在其中重新分配代价。

下面三个方案分别针对流量与公平。

[18] 视频画面时间区间：00:35:00–00:35:15。若为裁剪图，时间区间仍对应原视频画面。

6 三个改进方案

6.1 test-and-test-and-set

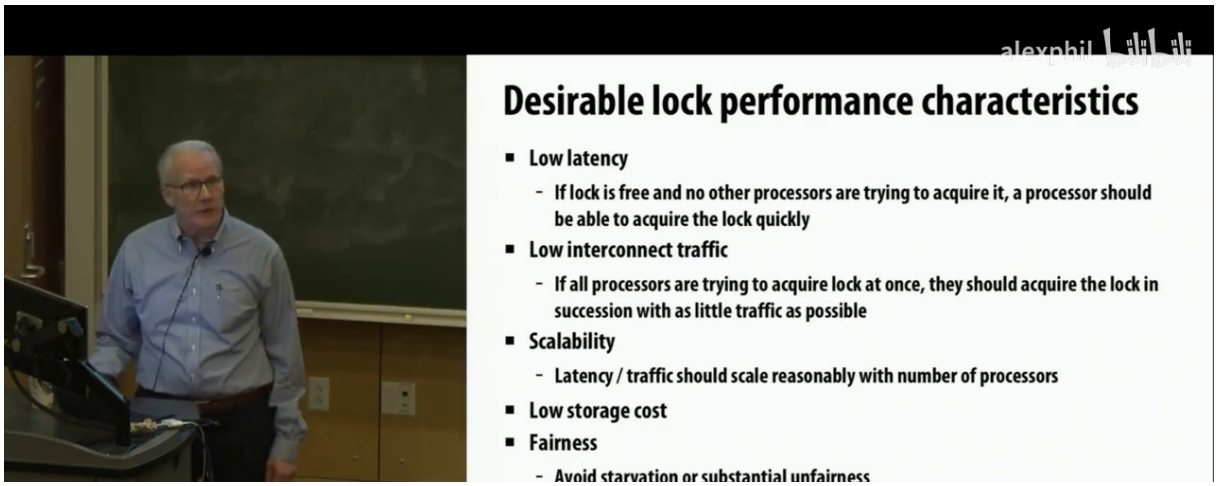


图 19: test-and-test-and-set：先只读自旋，看到空闲才尝试获取[19]

这个改进只有看过缓存协议才能体会其价值。与其反复猛敲 test-and-set，不如先在只读模式下忙等到锁变为零。

只读自旋不会作废他人的副本，因此几乎不产生流量。

一旦看到锁空闲，再用 test-and-set 去抓。若成功就完成了。

$$\text{流量}_{\text{TTAS}} \approx \frac{\text{流量}_{\text{TAS}}}{N_{\text{waiters}}}$$

- N_{waiters}：同时等待该锁的处理器数；
- TAS 每次尝试都要申请独占，成本随等待者数线性增长；
- TTAS 的自旋阶段只读，因此总流量降到接近单次交接的量级。

只读自旋利用了缓存共享状态

在 MSI/MESI 中，多个处理器可以同时持有同一行的共享副本。只读自旋让所有等待者停留在共享状态，不触发失效；只有真正尝试获取时才升级为独占。这把”等待”的流量从每次尝试降为一次。

[19] 视频画面时间区间：00:37:00–00:37:15。若为裁剪图，时间区间仍对应原视频画面。

6.2 退避

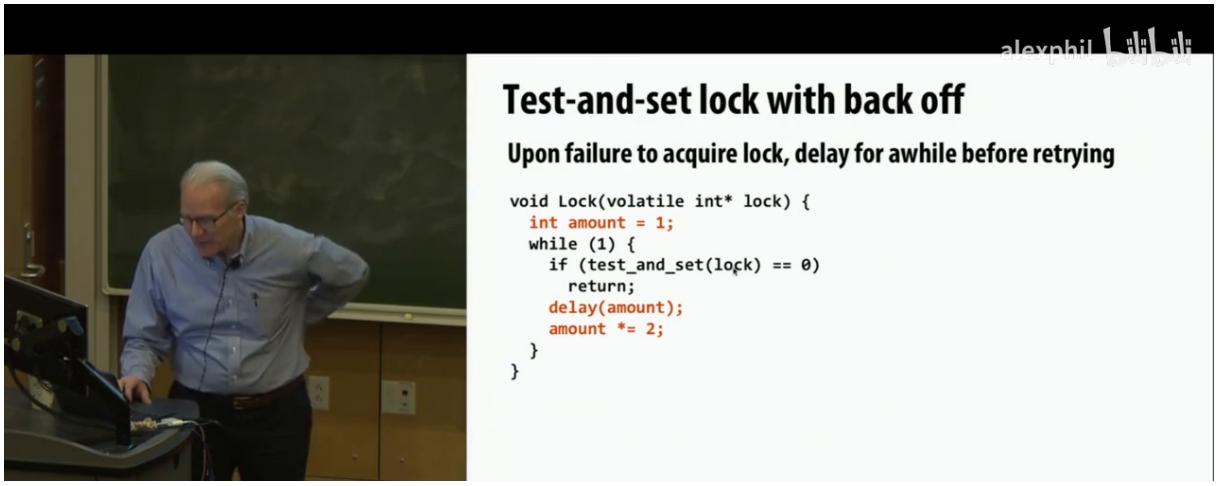


图 20: 退避策略：获取失败后延迟一段时间再重试[20]

第二个改进是退避：获取失败后，先等一会儿再重试。

讲者讨论了一个朴素版本的缺陷：如果我不断把延迟加倍，可能得到一个很大的数；而当第一个进程释放锁时，我还在等我的延迟到期。因此这个版本并不好，但基本思想是对的。

退避的另一个好处是改善交接时的可扩展性。特别是若延迟经过精心设置，有些线程会退后、有些会插入，它们不会一起扑上去。

退避参数必须与实际临界区长度匹配

退避时间过短则等待者仍会同时重试，起不到错开效果；过长则可能在锁已释放后仍在睡眠，浪费机会窗口。理想值取决于临界区平均长度与等待者数量，而这些在运行时变化，因此实际系统常采用自适应或指数退避并设上限。

6.3 票号锁



图 21: 票号锁：取号后按序获得服务，保证公平[21]

[20] 视频画面时间区间：00:42:00–00:42:15。若为裁剪图，时间区间仍对应原视频画面。

讲者介绍第三个方案，并说明它真正涉及操作系统层面的思考。票号锁也叫面包店算法，得名于在有些地方你确实会拿到一张实体票（现在多是屏幕上的版本）。

想法是：进入时先取一个号，服务方保证按号码顺序处理这些请求，因此你终会成为队首。

好处是公平——一旦拿到号，就保证在某个时刻会得到服务。

票号锁把竞争转化为排队

test-and-set 的问题在于所有等待者在释放瞬间同时争抢，谁抢到取决于时序而非等待时长。票号锁用两个计数器（取号与叫号）把这种竞争变成有序队列：等待者只需检测叫号是否轮到自己，因此既保证公平，也把交接流量降到每人一次读。

6.4 本章小结

三个改进各针对一个问题：test-and-test-and-set 用只读自旋把等待流量降到接近单次交接；退避用随机错开重试时刻缓解争抢，但参数需与临界区长度匹配；票号锁用取号排队保证公平并把竞争转化为有序检测。

这些方案都针对锁。屏障是另一类同步，它有自己的难点。

7 屏障与后续

7.1 集中式屏障

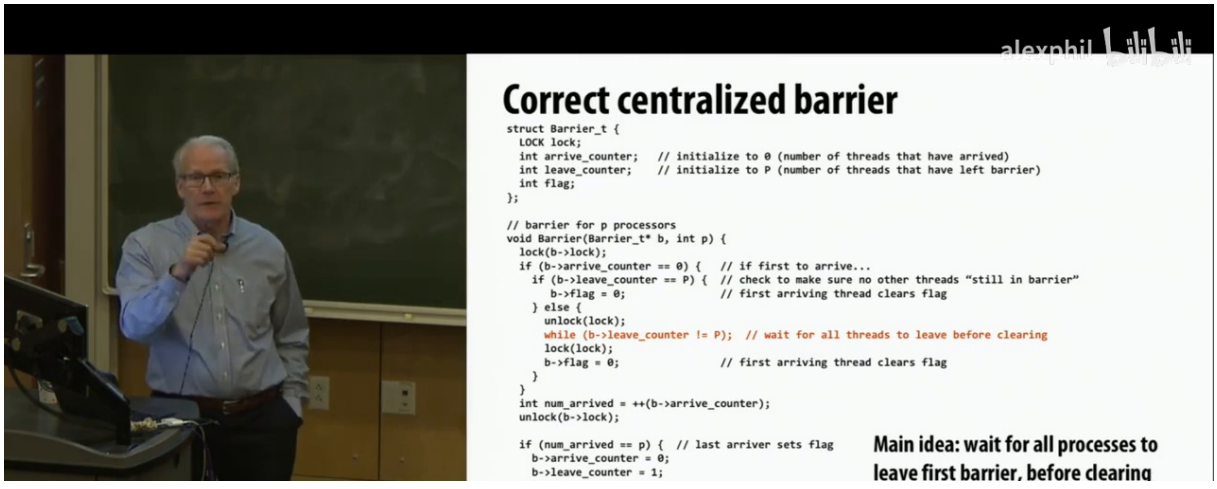


图 22: 集中式屏障：用一个锁与到达计数器实现[22]

讲者展示了一个正确的集中式屏障实现：结构里有一把锁、一个到达计数器，以及释放时用到的标志。

屏障的要求是：所有线程都到达后才能继续。因此实现需要计数——每个线程到达时递增，最后一个到达的线程负责释放其余所有等待者。

屏障的等待时间随线程数增长：

$$T_{\text{barrier}} \approx T_{\text{last}} + T_{\text{wake}} \cdot N_{\text{threads}}$$

[21] 视频画面时间区间：00:43:00–00:43:15。若为裁剪图，时间区间仍对应原视频画面。

[22] 视频画面时间区间：00:58:00–00:58:15。若为裁剪图，时间区间仍对应原视频画面。

- T_{last}：最后一个线程到达的时刻，取决于负载最重的那个；
- T_{wake}：唤醒一个等待者所需的通信代价；
- N_{threads}：线程数，唤醒是串行或广播的，因此该项随线程数增长。

屏障是”一对多”的同步，锁是”一对一”的交接

锁的语义是互斥：同一时刻只有一个线程在临界区内。屏障的语义是集合：所有线程必须都到达才算完成。后者需要区分两种角色（最后到达者与等待者），因此实现上必然比锁多一层状态管理。

讲者指出这比锁更复杂：锁只需让一个线程通过，屏障要让一批线程同时通过，因此需要额外的状态来区分”我是最后一个”与”我在等别人”。

屏障的代价随线程数增长

每次屏障都需要一次全局计数更新与一次全体唤醒。前者是串行点，后者产生广播流量。因此屏障的延迟大致随线程数线性增长，而它在迭代式并行程序（如每轮都要同步的模拟）中被频繁调用，往往成为可扩展性的主要瓶颈。

7.2 预览：事务内存

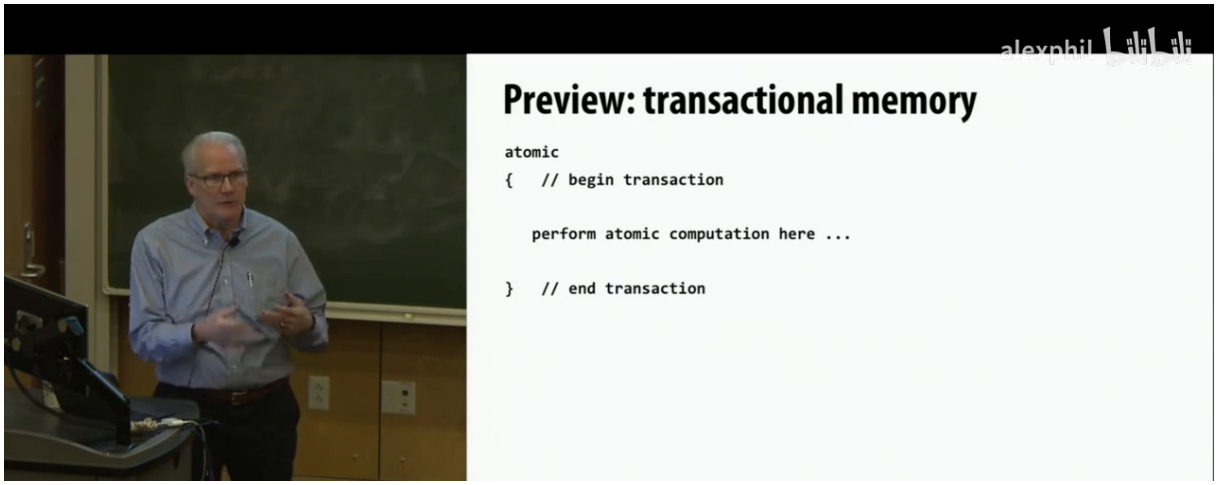


图 23：预览：事务内存，把一段计算声明为原子[23]

讲者在结尾预告后续内容：事务内存。它的想法是把一段计算声明为原子的，由硬件负责检测冲突并在必要时回滚。

他提到有意思的是，近年处理器确实加入了支持这类操作的特殊指令与能力。

7.3 本章小结

集中式屏障用锁加到达计数器实现，比锁更复杂，因为要让一批线程同时通过。它的代价随线程数增长，在迭代式程序中常成为瓶颈。事务内存把一段计算声明为原子，由硬件处理冲突，是后续讲座的主题。

[23]视频画面时间区间：01:05:00–01:05:15。若为裁剪图，时间区间仍对应原视频画面。

8 总结与延伸

把全讲串起来，这是一条从” 同步事件由什么构成” 到” 各种实现如何取舍” 的路径。硬件线程数远少于活跃线程数，操作系统负责映射；同步的开销量级取决于是否牵涉操作系统，纯硬件几十到几百周期，阻塞式数千周期。同步分获取、等待、释放三阶段。忙等待开销极低，在高并行计算中并不坏，因为等待时本无别事可做。朴素自旋锁用普通读写实现，测试与置位之间存在窗口，多个线程可能同时通过；实际出错概率因缓存协议开销而降低，但不构成保证。test-and-set 提供原子的读-测-写，但它每次尝试都要申请独占，解锁瞬间所有等待者一起争抢，产生大量流量，且不公平。理想锁需低延迟、低流量、低存储、公平，而 test-and-set 只在前两项之一与第三项上表现好。三个改进分别针对流量与公平：test-and-test-and-set 用只读自旋把等待流量降到接近单次交接；退避用随机错开重试时刻；票号锁用取号排队保证公平。屏障用锁加计数实现，代价随线程数增长。事务内存是后续方向。

本讲义在讲者材料之上做的压缩，有四点需要与讲者原话区分：

- 忙等待与阻塞的取舍取决于” 等待期间是否有别的事可做”，而不是取决于哪个更先进。并行计算程序通常用满所有核，因此自旋不浪费他人的执行机会；通用操作系统则总有别的线程可跑。脱离这个前提讨论优劣没有意义。
- 锁的四项特征互相冲突，因此不存在全面最优的实现。低延迟要求简单，公平要求排队，低流量要求错开。评价任何锁都必须指明竞争程度与负载模式。
- 原子性不能靠概率保证。朴素自旋锁在实际中很少出错，是因为缓存一致性协议的开销拉长了窗口两侧的时间，而不是因为它在逻辑上安全。处理器数增加或协议改变都可能让问题暴露。
- 只读自旋的有效性依赖缓存一致性协议的共享状态。若硬件不支持多副本共享读，这一优化就无从谈起。这说明锁的实现与底层一致性模型紧密耦合，不能跨平台照搬。

这堂课的边界在于它讲的是硬件层面同步原语的实现思路，未深入具体语言的内存序语义与事务内存的完整机制。讲者也说明真实实现远比示意版本复杂。

两个可以直接从本讲数据提出的下一步问题：其一，票号锁保证了公平，但它把交接变成串行的叫号传递，在处理器很多时延迟如何增长？其二，屏障的代价随线程数线性增长，那么在设计一个每轮都要同步的迭代程序时，应当增大每轮的工作量以减少同步次数，还是改用无屏障的算法结构？